- System concepts:
  - Introduction
  - Elements of system
  - System concepts
  - Types of system
- System Analysis
  - Role of System Analyst
- Software Engineering:
  - Definition
  - Characteristics of software
  - Qualities of software
- System Development life cycle
  - Process Models-
    - Waterfall model
    - V-Shape model
    - Spiral model
    - Prototyping
    - Incremental
    - RAD

****

System Concept:

Elias M. Awad defines a System as a systematic arrangement of the set of interrelated components that work together according to a plan to achieve a desired goal or objective i.e.(convert input into output). A system is accepting inputs and produces outputs. It is logically arranged by rules, regulations, principles, and policies. The system is created to solve the problems.

In the system, components are related to each other and work for hand in to hand achieve the desired goal. If the system is large then it divides into the subsystem.

Example:

1. Business System components like sales, marketing, research, accounting, etc.
2. Human body system components like the head, heart, hands, legs, etc. The various body parts are connecting each other via blood vessels, and nerves, and the main goal of the system is "living".
3. Computer System components like CPU, input device, output device, and users and their basic goal are to process the data.
4. Library System components like librarians, books, and periodicals provide knowledge to members.

Elements of a system

A system has three basic elements-input, processing, and output. Other elements of the system are control, feedback, boundaries, environment, and interfaces. They are in a fixed position which helps the system analyst to design & work with a system more easily.

- **Input**

Input may be data, information, raw materials, physical resources, or human knowledge. Inputs are supplied to the system for processing. The inputs must be put in the proper format because input defines the nature of the output. Input Data should be accurate. If the data is not accurate, the outputs will be wrong. If data is not obtained in time, the entire system falls into a loss. output of one the system is the input of another system

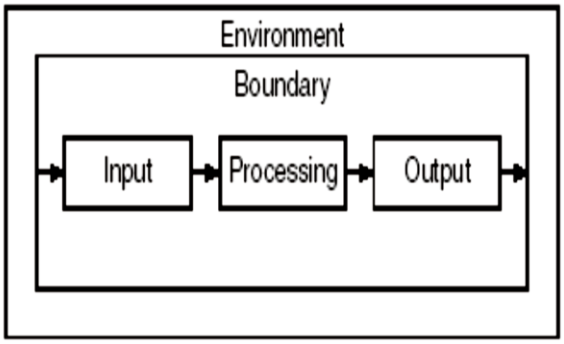
- **Processor**

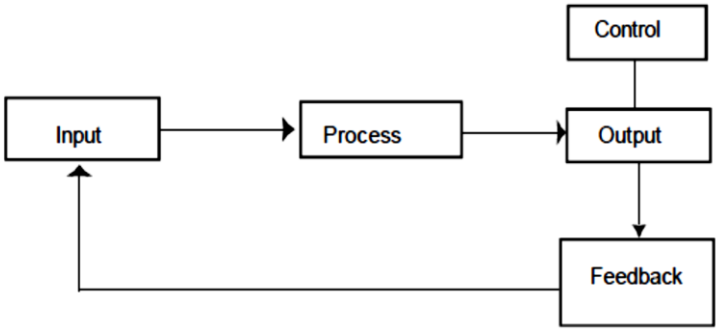
The processor converts actual input into output. It is a working component of a system. Processors may modify input totally or partially depending on the specifications of output. Man, machines, functions, operations, organizations, and combinations of these may act as a processor.

- **Output**

Output is the result of processing. Output depends on the input and its nature. The output format may differ from the input. A system feeds the input to produce output.

Example: Input data may be numerical but the output may be in pictorial diagrams like pie & bar diagrams.





To build any system on the basics of its elements knowledge, it does not possible. We need fundamental clarity which is necessary to build an efficient system. A model represents a system with its elements to keep a system in balance.

- **Control**

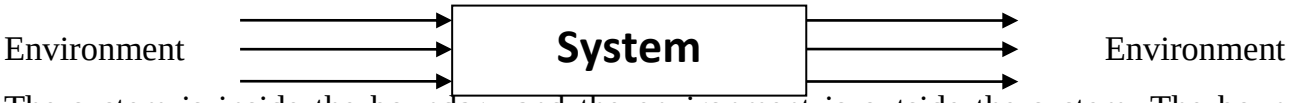
The control element guides the system. It is a decision–making subsystem that controls the pattern of activities governing input, processing, and output.

System Concepts

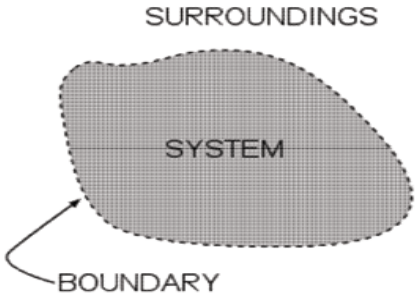
- Boundaries and Environment
- Subsystem
- Interface
- Black-Box
- Feedback Control

➢ Boundaries and Environment

Boundaries are the limit of the system. Setting up boundaries helps to identify a system's components, processes, and interrelationships when it is connected with another system.



The system is inside the boundary and the environment is outside the system. The boundary of a system may be physical or conceptual.

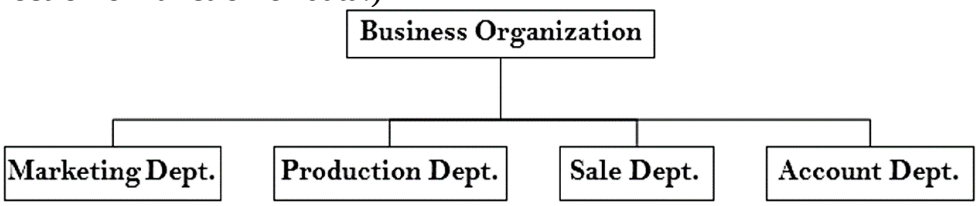


Everything outside the boundary of the system is known as Environment. Change in the environment affects the working of the system. Flow from the environment to the system is input and flow from the system to outside is output. The system defines boundaries.

➢ Subsystem

A complex system is difficult to implement. The system is divided into subsystems. These

subsystems are again divided into smaller subsystems that are manageable in size i.e. module (**Module:** A collection of function or data.)



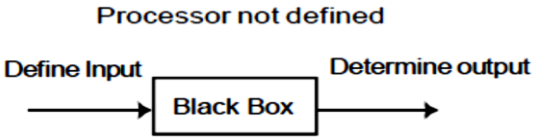
Example: In a business organization system, marketing, production, and sales can be considered subsystems.

- **Interface**

The interconnections and interactions between the sub-systems are known as the Interfaces. They may be inputs and outputs of the systems. Each interface indicates a communication path. The number of interfaces increases with the number of subsystems.

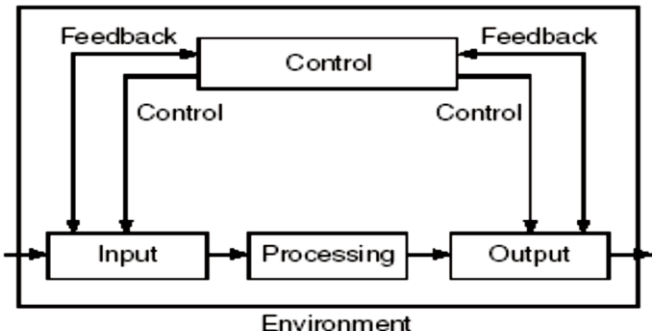
- **Black box**

In the Black box, inputs are defined, outputs are determined but the processor is not defined. It was difficult to understand how the conversion of input to output.



- **Feedback control**

Feedback is the data about the performance of the system. Control is a system function that monitors and evaluates the feedback to determine whether the system achieves its goal or not. It controls the input of the system after analyzing the output.



Feedback can be positive or negative.

- **Positive feedback-** A system performs towards its goal.
- **Negative feedback-** if it is falling from its goal. It will help us to improve the quality of output.

Types of System

- Physical System and Abstract System
- Open System and Close System
- Deterministic System and Probabilistic System

- **Physical System:**

Physical systems are tangible entities that we can feel and touch. Physical systems are made of physical entities, people, materials, machines, energy, etc. These may be static or dynamic. Static systems don't change. Dynamic systems are constantly changing.

Example: A computer center. Desks and chairs are the static parts, which help in the working of the center. Computer systems are dynamic systems. Programs, data, and applications can change according to the user's needs. Construction of the building, Dam, automobile system, etc.

- **Abstract System:**

Abstract systems are composed of ideas, policies, and theories and conceptual or intangible. These

are not any physical entities. They are concerned with theoretical structures that may or may not exist in real life. They are also called conceptual systems. **Example:** A system of economic theory, a system of scientific theory or Concept, an algorithm or an equation, etc.

- **Open System:**
  An open system interacts with its environment by receiving inputs from outside and sending outputs outside the system. Almost all systems are open because they are affected by the environmental effects.
  Examples: Recruiting System, Production System, Marketing System, our body system, educational system, business system, and all other systems which take input and return output. Also, Information systems are open systems since they accept inputs from the environment, and produce output.
- **Close System:**
  A closed system does not interact with its environment. It is isolated from environmental effects, it is not affected by changes occurring outside the system. Closed systems are more related to scientific systems.
  Example: A chemical reaction in a sealed, insulated container is a closed system. A computer system is a relatively closed system.
- **Deterministic System:**
  It is a system that operates in a predictable manner and interaction between parts is known with certainty. Stepwise execution is always possible & output is sure. **Example:** The output of a computer system is deterministic. Two molecules of hydrogen and one molecule of oxygen make water
- **Probabilistic System:**
  It is a system that operates in an unpredictable manner & degree of error is always possible. Also, the output is not sure.
  Example: Economic forecasting, mail delivery
- **Natural systems:** All the naturally occurring systems are called natural systems
  Example: Solar system, Water System.
- **Artificial system:** All manmade systems are called artificial systems. **Ex:** Motor Car.

System Analysis

System analysis is the process of gathering information, interpreting facts, and identifying problems. Using this information to recommend improvements in the system. System analysis is concerned with:

1. Investigating
2. Analyzing
3. Designing
4. Implementing
5. Evaluating information systems in organizations.

<u>A system analyst is a key person who is the brain of the system development life cycle. He acts as an interface between the user (client) and the organization.</u>

A system analyst is not only a programmer of the system but also more than a manager.

I. Defines the design of the overall system
II. Takes the necessary technical help from programmers, specialists, and engineers.
III. Follows the system through design, implementation, and evaluation activities.

Systems Analyst

System analyst guides the system development process. The tasks of the analyst must always match the system objectives with the goals of the organization. A system analyst is a multitasking personality. He has to perform different roles at different stages of the system development.

Function or Task of System Analyst:

The primary role of a systems analyst is to study the problems and needs of the organization. They

determine how people, methods, and technology can be combined to bring improvements to the organization. A systems analyst helps system users and business managers define their requirements for a new or enhanced system.

Following are the tasks performed by the system analyst:

- Collects the facts of the existing system.
- Analyses the collected information, basic methods, and procedures of the current system
- Determines and specifies the needs
- Designs the information system
- Prepares the model
- Modify redesigns, verifies, and integrates according to required specifications.

The main objective of a system analyst is to provide the right type of information at right time in the right quality and quantity in the right way and at the right cost to the management and the client.

Role of System analyst

System analyst has to perform many roles such as

1. Architect
2. An agent of change
3. Investigator & motivator
4. Organizer
5. Motivator & Psychologist
6. Salesperson

1. An architect:

An analyst is the creator of the physical design of the system. The analyst creates a detailed physical (current) and logical (proposed) design of the system. A systems analyst makes the design of system architecture based on end-user requirements. This design becomes the blueprint for the programmers. He is a brilliant architect of the system.

2. An agent of change:

System analyst works for an uncertain future. So analyst has to prepare a new model for this changing environment. It is important to user accept change to secure user acceptance through user participation in the development process. Systems Analysts may use different approaches to introduce changes to the user organization. Analysts carefully implement changes into the user domain because people naturally oppose changes.

3. Investigator & monitor:

The analyst studies the problems and needs of an organization during the feasibility and requirements analysis phases. He analyses the problems of the current system and collects their new requirements. The analyst initially works as an investigator by extracting the real problems of the users.

Monitor: Monitor is to undertake and complete a project. Analysts must monitor programs with time, cost, and quality. He can control the flow of the system and thereby monitor the system as a ‘whole’. If time "gets away", the project suffers from increased costs and wasted human resources.

4. An organizer:

The analysts should have a clear idea of all the activities of the system. He should be able to put all the activities in sequence & clear about their purpose. He is responsible for the execution of the activities and result. He is an evaluator of the system.

5. Motivator & psychologist:

The analyst motivates users to participate in the development and implementation of the proposed system. This helps to understand the user’s feelings about the proposed system.

System acceptance is achieved through user participation in its development, effective user training, and proper motivation to use the system.

Physiologist: systems are developed by people. Psychologist reaches people, interprets their thoughts, assesses their behavior, and draws conclusions from these interactions. Psychologist plays a major role during the phase of fact-finding.

6. Salesperson

Selling change can be essential as initiating change. Selling the system takes place at each step in the system life cycle. Sales skills and smoothness are important to the success of the system.

Software Engineering:

Software is a set of instructions that contains executable programming code, libraries & documents. When S/W is made for a specific requirement is called a **software product. Engineering** is all about developing products using well-defined scientific methods, and procedures with principles.

Software engineering is the development of software products using well-defined scientific methods or procedures with principles. The outcome of software engineering is an efficient and reliable software product.

Software Engineering involves a "systematic approach to the Requirements, analysis, design, code, test, implementation, maintenance and reengineering of software".

"The main goal of s/w engineering is to understand customer needs and develop a software product with better quality, on time, and within budget.

Need for Software Engineering

The need for software engineering arises because of a higher rate of change in user requirements and environment.

- **Large software:** If the size of the software is large, then it required a scientific process for development.
- **Scalability-** A software process isn't based on scientific and engineering concepts, it would be easier to re-create new software than to scale an existing one.
- **Cost-** the cost of software is increased if the proper process is not used.
- **Dynamic Nature-** The nature of software is always changing, new enhancements need to be done to the existing one. Software engineering plays a good role.
- **Quality Management-** Better software development process provides better quality software

Software Characteristics:

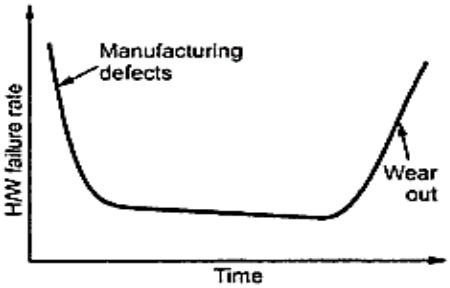
To examine the software product design process. A software product differs from a hardware product. Its clear software is a logical rather than a physical system.

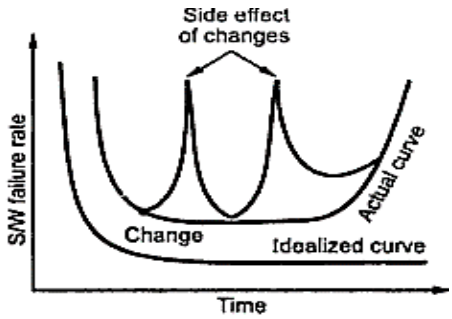
- **Software is developed or engineered, not manufactured**
  Software development and hardware development are two different activities. A good design is the backbone of both development activities. When quality problems occur in the hardware manufacturing process cannot be removed easily. But in the software development process, such problems can be repaired. In both development processes, developers are responsible for producing a quality product.

- **The software does not wear out**
  In hardware development, the process failure rate is very high because of manufacturing defects. But after correcting defects failure rate gets reduced. After some time again failure rate becomes the same or increases because of environmental problems i.e. extreme temperature, dust, vibrations, etc.

  Software is not destroyed by environmental changes i.e. It does not wear out. But changing user requirements may create some defects in software, and for this reason, the failure rate is high. Before the curve can return to the original steady state (quite low) another requirement change is

requested then the failure rate becomes high. Thus frequent changes in software cause to failing the quality of software and the failure curve look like a spike. Failure rating of software *"actual curve"* is as shown below.





The software requires proper maintenance, which reduces the failure rate by correcting the error. It needed an "idealized curve".

If the hardware component wears out it can be replaced by another component but it is not possible in software because the software has no Parts. Software maintenance is more difficult than hardware maintenance.

- **Most software is custom-built, rather than assembled from existing components**
  A software component should be designed and implemented so that it can be reused in many different programs. The software development approach is use reusability of software components. Example: GUI is built using reusable components such as message windows, pull-down menus, and many more such components.

Software Qualities:

Quality of software measured by defect-free, deliveries on time, within budget, meets user requirements, and expectations. Software systems are delivered to the user with documentation that describes the features of the products, how to install them, how to use them and how to maintain them etc. There are many desirable software qualities. The software product quality must be

(User's view / External Qualities)	(Developers view / Internal Qualities)
Reliable	Verifiable
Efficient	Maintainable
Easy to use.	Portable
	Extensible
	Productive
	Easy to control.

<u>Another way of classifying quality is</u>

Product Quality	Process quality
Maintainability	Understandability
Dependability	Visibility
Efficiency	Supportability
Usability	Acceptability
	Reliability
	Robustness
	Maintainability & Rapidity

➢ Correctness

Correctness is the ability of s/w product to perform the required function, and tasks as defined by their specification. Correctness is measured in KLOC (defect per thousand lines of code) where defect means verified lack of matches to requirements. "Software correctness" means a combination of accuracy and reliability.

➢ Robustness

Robustness measures how the software system reacts to or deals with unexpected events, input, or conditions that occur in the Working environment. Unexpected events like software crash, disk crash,

incorrect input data, power-off, etc. Robustness borders on correctness & reliability. Once invalid inputs or conditions are identified, then the system takes strong action against unexpected events, inputs, or conditions. Example: Antivirus

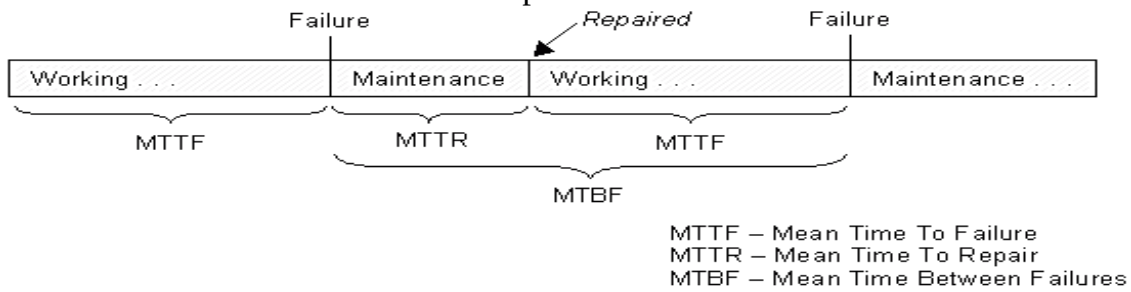
➢ **Reliability**

Software Reliability is the probability of failure-free software operation for a specified period in a specified environment. It also determines the correctness of the program. An ability to recover from a failure and calculate the time between failure and repair.

Suppose, a program 'X' runs 100 times and works 96 times without failure in 8 hours of execution time. i.e. program reliability 0.96 over 8 hours.

Measure of reliability is: **MTBF = MTTF + MTTR**

MTBF is more useful to measure S/W as compared to defects/KLOC.



➢ **Performance**

Performance is measured by processing speed, response time, resource consumption, throughput, and efficiency.

- If a software system is too slow, it reduces productivity.
- If a software system uses too much disk space, it may be too expensive to run.
- If a software system uses too much memory, it may affect the other applications that are run on the same system.

Performance effects on the scalability of a software system. Our effort will be to increase the performance of S/W and minimize the need for redesign. Performance is closely related to productivity.

➢ **User Friendliness**

User-friendly defines a good software interface i.e. easy to use. A good user interface is well-organized, making it easy to locate different tools and options. If the software is not user-friendly then it is a failure. The user interface is an important component of user-friendliness. It can be measured by the following characteristics.

- ✓ A little bit of knowledge is required to learn the system.
- ✓ The time required to become fairly efficient in the use of the system.
- ✓ To increase productivity.
- ✓ Assessment of the user approach to the system.

➢ **Verification (Verifiability) and Validation (V & V).**

Verification will help to determine whether the software is of high quality. Verification checks whether the system is well-engineered and error-free. Verification is verifying documents, design, code, and program. According to Boehm - **Verification: "are we building the product right?"**

Verifiability (Also called testability) is the measure of effort required to

1) Specify the validation criteria for a product
2) Verify that product meets its validation criteria.

For example: an S/W design is harder to verify than a mathematical Function because the criteria for establishing the correctness of both products are different. A mathematical Function is also easier to test with known values.

Validation: "Are we building the right product?"

Validation is the process of evaluating the final product to check whether the software meets the customer's expectations and requirements. This is a dynamic mechanism for validating and testing the actual product.

➢ **Maintainability**

The effort required to fix and locate an error in a software product is called maintainability. i.e. bug fixing. Maintenance is a wide range of activities that include corrective, adaptive, preventive, and perfective maintenance. The system also can upgrade without interrupting the current records. Also, maintainability has two separate qualities-

- **Repairability-** software system is repairable if it allows the correction of defects with a limited amount of work.
- **Evolvability-** it is a process & product-related quality, software products are modified over time to provide new functions or to change existing functions. Software is so flexible for modifications & extremely easy to implement.

➢ **Reusability**

Reusability is the measure of effort required to reuse a component or object in a System. It specifies how to program can be reused. Reusability in the software industry rare service but this day's situation is slowly changing. The need for reusability in software, reuse existing modules instead of developing a new module. It saves production costs and time.

The reuse of software can be measured at several different levels.

- Application system reuse
- Subsystem reuse
- Object or module reuse
- Function reuse

➢ **Portability**

If a software system work on a different hardware and software platform such as operating system, compiler, etc. It is important to try to make all products as portable as possible because a software product has a longer life than hardware. Software needs to be ported from one H/W configuration (one machine) to another (machine) S/W system environment.

➢ **Understandability**

Software systems are easier to understand than other systems. But some tasks are more complex than others. To follow certain guidelines to produce more understandable designs and programs. Example- A weather forecasting system will be harder to understand than printing a mail. Understandability has internal product quality such as evolvability and verifiability and external product quality such as user-friendliness.

➢ **Interoperability**

Interoperability is the ability of software to exchange information, data, and services with other applications or systems (systems to coexist and cooperate with other systems). i.e. one system connects to another system. It calculates the effort required for transferring a program to another system. Interoperability is increased with the use of good standard interfaces.

Example- In android mobile, we can send a photo from the gallery application to the WhatsApp application.

➢ **Productivity**

It measures the efficiency of the process. An efficient process gives results in faster delivery of the product. Basili and Zelkowtiz define five important factors that control software productivity.

- People Factor - The expertise in software development.
- Problem Factors - the complexity of the problem.
- Process Factors - analysis and design techniques used.
- Product Factors - reliability and performance of the system.
- Resource Factors - availability of hardware and software.

All factors are needed for software productivity.

➢ **Timeliness**

It measures the ability to deliver a software product on time. Lack of timeliness in the software production process leads to "software crises". Timeliness requires careful scheduling, accurate work

estimation, and specified and verifiable milestones. Timeliness is addressed by cost estimation models. The standard project management technique-(PERT / CPM) is difficult to apply because of difficulties calculating the required amount of work, the productivity of the engineer, and continuously changing user requirements.

To overcome this difficulty, Timeliness can be achieved through the incremental delivery model. i.e. a product is divided into an incremental process.

Example- Word Processing software,

- editing and document production functions in the first increment
- more sophisticated editing and document capabilities in the second increment
- spelling and grammar checking in the third increment
- Advanced page layout capabilities in the fourth increment.

➢ **Visibility**

In the software development process, all steps must be visible and their current status documented. Visibility means "Transparency" or "Openness". Visibility helps to evaluate software engineers and managers to know the exact status of the product and the risk of serious problems and delays.

If a key engineer leaves the project then new engineers will immediately understand the tradition of the project. Lack of visibility reduces productivity and affects the timeliness of the project.

➢ **Flexibility**

Flexibility is a measure of the effort required to extend the functionality of the software or modify an operational program (Also called extendibility, Evolvability). Changes in the software should be easy to make. If customers required some future enhancements. These enhancements can be input into the design phase because of flexible design. Flexible software design supports an extension of the system. Developing iteratively also supports the flexible design, since each iteration is effectively a product extension.

➢ **Efficiency**

Efficiency is the ability of a system to perform certain tasks within a time frame required by the user. It is calculated by assessing processing speed, response time, resource consumption, throughput, and efficiency. Efficiency is an internal quality and performance is an external quality.

➢ **Integrity:**

The efforts required for unauthorized access of software or data are controlled. To its extent, unauthorized access to the software can be prevented. It determines whether or not the software takes security.

✍ System or Software Development Life Cycle (SDLC)

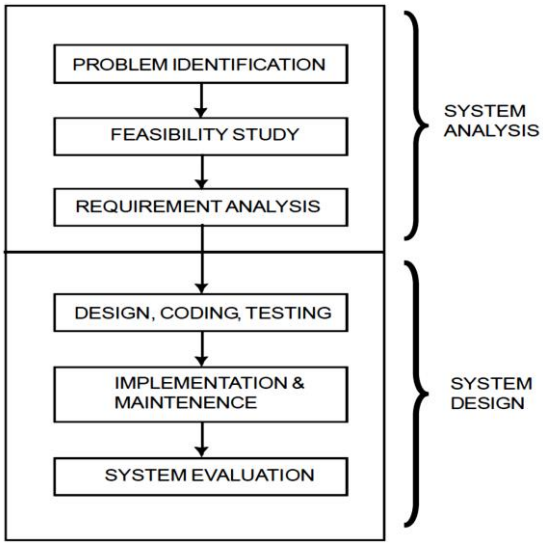
SDLC is a systematic approach that creates a structure for a developer to develop and delivers high-quality software according to customer requirements. The purpose of the SDLC process is helping to produce cost-efficient and quality products. It also provides guidance, policies, and methodology. The number of phases in the system development may vary from model to model. Each phase gives an output that becomes the input of the next phase. He defines a framework for software development through plan, analysis, design, implementation, and maintenance. SDLC is a well-defined process by which the system can be visualized, developed & implemented. SDLC phases are

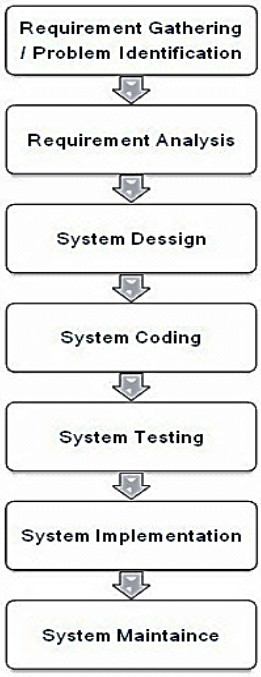
➢ **System Analysis**

1) **Problem Identification**
2) **Feasibility study and Cost Benefit Analysis**
3) **System requirement analysis**

➢ **System Design**

1) **System design specification and programming**
2) **System testing ,implementation & follow-up**
3) **System maintenance &Evaluation of the system.**





System Analysis

System analysis means the identification & study of the system & its parts to achieve the objective through modifying interrelationships of components, and deleting or merging components. It may upgrade of system.

Requirement gathering and analysis:-Preliminary Investigation OR Problem Identification

To collect the requirements like input, output data and use of the system through arrange meetings with managers, stakeholders, and users.

Problem identification is the process of identifying the problem of the existing system and determining why a new system is needed and identifying the objectives of the proposed system. If the problem is not properly identified it may lead to wastage of time and energy. Problem identification the focus is on:

1) Source of the problem: starting place of the problem may be internal or external.
2) Types of the problem: problems in the system are related to software Quality.
3) Nature of problem: find out the exact nature of the problem such as how complex it is and Likely solutions.

Problem Identification helps:

1. It helps in setting proper system goals and objectives
2. Determines the boundary and scope of the system.
3. Checks limitations of available resources.

Feasibility study and Cost Benefit Analysis

It defines the possibility of either improving the existing system or developing a completely new system. It helps to obtain an overview of the problem and to get a rough assessment of whether a physical solution exists. The purpose of a feasibility study is to determine whether the requested system is successfully possible and evaluate the cost & benefits of the system. A detailed feasibility study was conducted by Steering Committee. The committee has system analysts, representatives from departments, and clients.

Major aspects of feasibility study:

- **Economical:** Can we complete the project within the budget or not?
- **Operational**: Can we create operations that are expected by the client?
- **Technical:** Need to check whether the existing computer system can support the S/W.
- **Schedule:** Decide whether the project can be completed within the given schedule or not.

System Requirement Analysis

System analysis defines 'What is to be done. Requirement analysis is a determination of requirements for a new system. System Analyst defines all the requirements of a system known as Software Requirements Specifications. The SRS is validated by the users by reviewing their requirements

specified in the analysis. The analysis phase is a detailed investigation of operations, the relationship among functional components, and the boundary of the system. Many tools are used to assess the user requirements such as Interviews, Questionnaires, Developing various flow charts and Observations, etc. This will make the system more users friendly rather than designer friendly. Requirement analysis will determine

1. What outputs are needed?
2. What inputs are needed to obtain these outputs?
3. What operations it must perform to obtain these obtain?
4. What resource must be used?
5. What operational and accounting controls are needed?

Requirement Statement specifying enough details about the new system requirements.

System Design

System Design defines How it is to be done. System design is the most challenging and creative phase of SDLC. Transform the SRS document into a logical structure, which contains detailed and complete specifications that can be implemented in a programming language.

In the system design phase, the system is divided into different modules and each module is designed. System design includes identification of input data and output reports, design of input forms and output forms, database, interfaces, and architecture. The major objective of system design is to build the best model of the system that satisfies user requirements.

Several tools and techniques are used for describing the system design of the system. These tools and techniques are Flowchart, Data flow diagram (DFD), Data dictionary, Structured English, IPO (Input Processing and Output) and HIPO (Hierarchy of IPO) charts, Decision table, and Decision tree.

Coding:

Coding is translating system design into code using the chosen programming language. Tasks are divided into units or modules and assigned to the various developers. It is the longest phase of the SDLC process. The developer follows certain predefined coding guidelines, the layout of programs, and naming conventions for variables, classes, and functions. Developing programs that are easy to read and understand. Structured programming helps to make programs understandable. A well-written code can reduce the testing and maintenance effort and otherwise affect both phases.

Testing:

It is impossible to deliver quality software products without testing. Testing is useful to detect errors in the software. The goal of testing is to find coding errors and uncover requirements in the S/W. Testing is a very critical and time-consuming activity. During the testing, the actual result was compared with the expected output. The final output of the testing phase is the test report and error report. During the testing phase importance-

- Testing should be planned before starting.
- All tests trace whether customer requirements are fulfilled or not
- Code quality
- Testing is done by different types like functional, Non-functional, performance, system, integration, and acceptance testing
- More effective testing should be conducted by an independent third party.

The output of the testing phase is functional software, ready for deployment in the environment.

Implementation

In an implementation, the system is installed at the user's place and implemented. It involves programmers, users, and operations management. The manual or old computerized system is converted into a newly developed computerized system. Different types of conversion methods are used like direct cutover, parallel run, pilot, and phase-in.

It consists of installing hardware, and programs, collecting data, and running the system. Users are trained for operating and maintaining the system. Implementation includes

- Site preparation and Planning of implementation

- Purchase Hardware and software.
- Training to end users who will operate the new system.
- Users training, meetings & seminars
- Conversion of data files from the old system to a new system.

Maintenance:

After the new system has been implemented, Maintenance is necessary to remove errors and problems during the working system and to adjust the system in any working environment. The development team must maintain the software by fixing bugs and adding new features. It takes a review of the system from time to time. During the maintenance phase:

- Bug fixing - bugs are reported because some developers are not tested.
- Upgrade - Upgrading the application to the newer versions of the Software
- Enhancement - Adding some new features to the existing software

System maintenance is an ongoing process. Maintenance is the most expensive as it uses more energy, cost, and time and it leads to the birth of a new system.

Types of maintenance:

1. Corrective maintenance: bugs (errors) are corrected

2. Adaptive Maintenance: certain modifications are incorporated into the system

3. Perfective maintenance: Enhancing performance and modifying the software to improve some of its qualities. For example usability, user-friendliness, etc.

4. Preventive Maintenance: prevention of failures of equipment before it occurs.

Evaluation of the System:

Evaluation means feedback on the system. Evaluation is the process of verifying the capability of the system after operation to check whether users' objectives are fulfilled or not. It includes response time, overall reliability, and limitations on user behavior. Evaluation is done to estimate the weakness and strengths of the system.

Differentiate between system analysis & system designs.

system analysis	system designs
System analysis is the assessment of the problem.	System design is the solution to the problem.
It deals with data collection and a detailed evaluation of the present system	It provides technical specifications & reports with which the problem can be tackled.
Performed by a Systems Analyst	Performed by a Software architect
what the proposed system will do	how it has to be built- database design, I/O design, and user interface design to meet the *system specification*

Process Models

Software development is a layered technology that is based on its commitment to quality. It uses tools and techniques to give high-quality software products. Different software development models such as the waterfall model, spiral model, prototyping, and many more.

Waterfall Model

Waterfall Model is a **linear-sequential life cycle model or Classical model** that divides the software development process into different phases. It was introduced in 1970 by Winston Royce. This model is used for small projects and they have fixed requirements. One phase can be started after the completion of the previous phase. That is the output of one phase will be the input to the next phase. There are standards set for every phase output. If the output of any phase is wrong, it may affect the following phases and disturb the entire model. The waterfall model is also useful to measure the progress of the project.

1. **Preliminary investigation:** All the detailed requirements are gathered from the customer who wants to develop the software. To find out the source of the problem of the system.

2. **Requirement analysis:** system analyst analyses the requirements and whether requirements can be fulfilled or not. If customer requirements can be fulfilled within budget or not.
3. **System design:** Requirements translate into design. System architect designs software using different diagrams such as I/O Design, Procedures, class diagram, data flow diagram, state transition diagram, etc. When a team is completely satisfied with the proposed design, then move to the next phase.

4. System coding: software design is translated into source code using any suitable programming language. Thus each designed module should be coded.

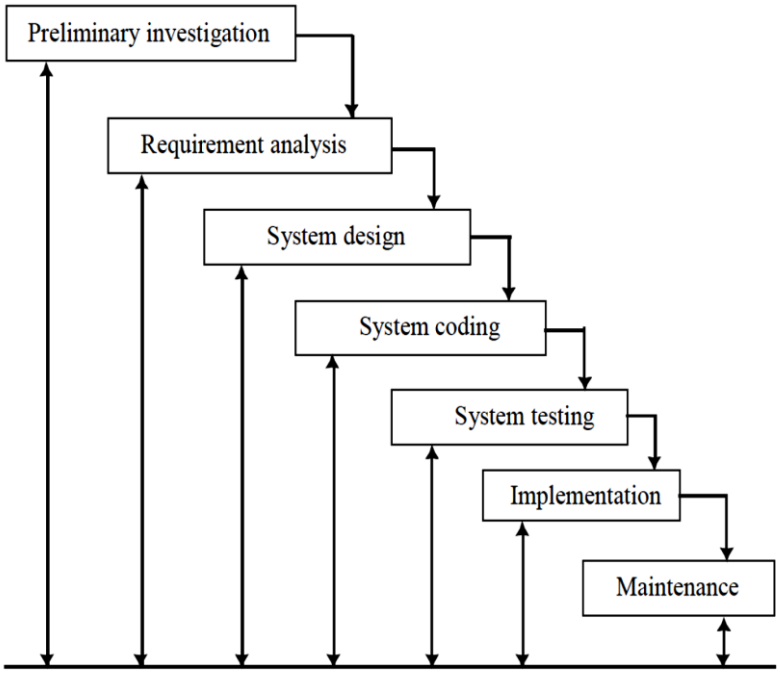


Fig : Waterfall Model

5. System testing: we test each unit or component and make sure the developed components are working as per customer requirements. Testing identifies and removes errors from the program. All the types of testing are done in this phase.

6. Implementation: the system is delivered & installed at the customer's end and provides support. Feedback is taken from the customer to ensure the quality of the system.

7. Maintenance: Maintenance is a phase that allows modifying the code to improve the quality of a product. The modifications occur due to change requirements or adding new functionality started by the customer. Restructuring complete code increases the cost of maintenance.

- **Advantages of waterfall model:**
  - Easy to understand, manage and use.
  - Systematic & sequential model
  - Each phase has deliverables-specific output.
  - Phases are processed one at a time.
  - Cost-effective.
  - Phases do not overlap.
  - Used for smaller projects where requirements are Fixed & clear.
- **Disadvantages of waterfall model:**
  - Not useful for a large and ongoing project.
  - Less effective if requirements are not clear at the beginning.
  - Very difficult to move back to change from the previous phase.
  - High risky.
  - Less flexible.
  - Customer interaction is less because Customer feedback is only at the end of the product.
- **When to use the waterfall model:**
  - Use only when a project is small and requirements are very clear and fixed.
  - Product definition is stable.

- Technology is understood.
- For low-budget projects.

V-Process Model (Verification and Validation Model)

The major disadvantage of the waterfall model was testing done at the end of the development life cycle. It became very challenging & costly to fix the defects after the development process.

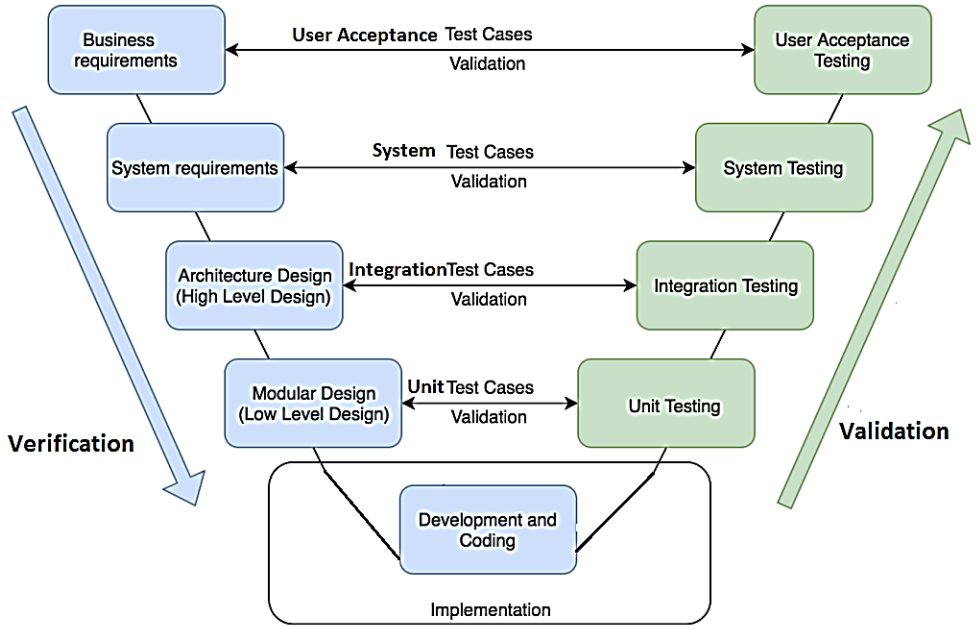
To overcome this problem, a new development model was introduced in 1980 by Paul Rook called the "V Model". V-Model is an extension of the classic waterfall model because Testing of the system is planned in parallel with a development phase. It was developed to improve the efficiency of software development. This model is processed step-by-step in which the next phase starts only after the current phase is verified and complete. In this model, testing starts at the very beginning i.e. from the requirement phase. Each phase of the development life-cycle has to prepare a parallel test plan for the testing of the system. The left-hand side shows development activities and the right-hand side shows testing activities. Test levels show by lines between both activities.

- **Verification:** Verification is a static analysis technique where testing is done without executing code. Examples- Reviews, Inspection, and walkthrough.
- **Validation:** Validation is a dynamic analysis technique where testing is done by executing code. Examples-functional and non-functional testing techniques.

Development and Testing activities are done simultaneously.

1. Requirements Gathering and Analysis:

- **BRS-** Business Analysts will collect complete detailed requirements from the client and analyze them. BA creates (BRS) Business requirement specifications. Both test and development teams start their activities. The testing team will write a user acceptance Test Plan for review BRS for finding any missing or wrong requirements.
- **SRS-** Manager and Analyst translate BRS into SRS(Software Requirement Specification) Using SRS, gathering the requirement required for a system build. The testing team will review the SRS against the BRS and writes the system test plan and test cases.



2. System Design: Once you have the clear and detailed product requirements,

- **Architectural Design** (High-level design)**:** An architect divide system design into components having different functionality. It consists of the list of modules, brief functionality of each module, their interface relationships, dependencies, database tables, architecture diagrams, technology detail, etc.
- **Module Design** (Low-level design)**:** Entire system splits into smaller modules. The detailed design of system components gets written down with greater clarity. It defines the actual logic for every component of the system.

3. Coding: Module design is converted into code by developers.

4. Implementation: Once coding is complete, the path of execution continues up the right side of the V where the test plans were developed earlier to use.

Validation phases are on the right side of the V-shape. It consists of:

- **Unit testing:** testing at the code level and removing the bugs at the beginning time.
- **Integration testing:** separate modules will be tested together to expose faults in the interfaces and the interaction between integrated components.
- **System testing:** System testing will compare the system specifications against the actual system. It checks if the integrated product meets the specified requirements.
- **User Acceptance testing:** Ensures that the delivered product fulfills the client's requirements.

Advantages of V-model

- This saves a lot of time. Hence a higher chance of success over the waterfall model
- Testing is planned parallel to the development.
- Bugs detection in the early phase.
- It provides better management of project risks
- Easy to Understand and manage.
- Works for small projects where requirements are fixed.

Disadvantages of V-model

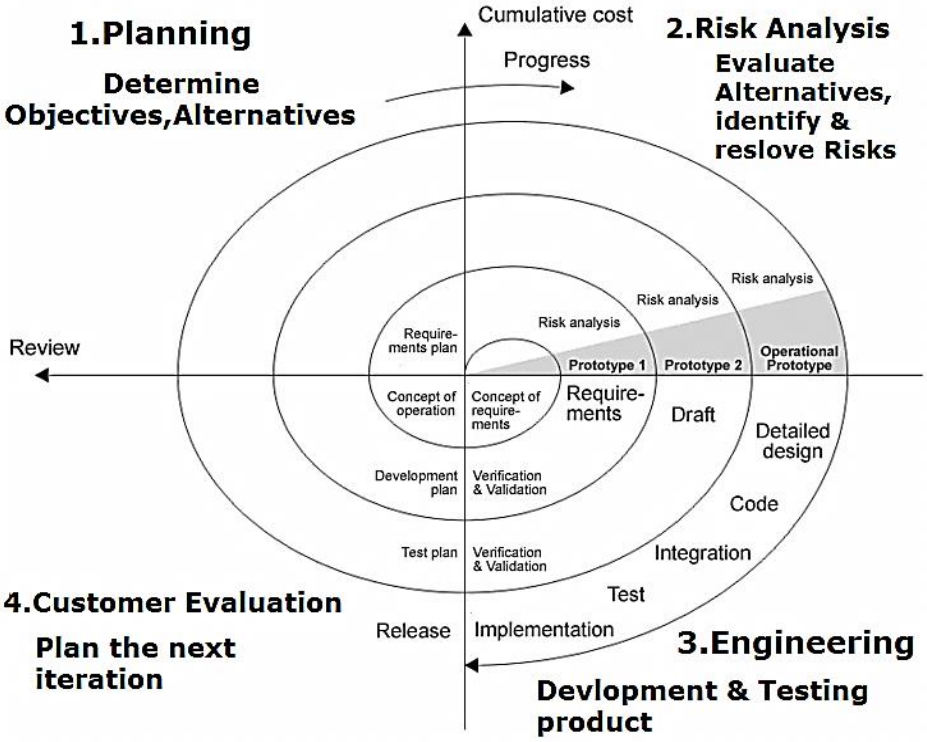
- Very strict and least flexible.
- Not suitable for bigger and more complex projects
- The expensive model as compared to the waterfall model i.e. required a lot of resources.
- V Model doesn't provide a clear path for problems found during the testing phases.
- Adding or changing requirements in the middle of the process is difficult.

When to use V-model

- Used for small or medium-sized projects where requirements are clear and fixed.
- When sufficient technical resources are available with needed technical expertise.

Spiral model

The problem with traditional SDLC models is that they do not sufficiently deal with the project risk. Software projects failed because project risks were neglected. Nobody was prepared when something unexpected happened. Barry Boehm tried to remove the "project risk" factor into a spiral model presented in 1986. Spiral Model is a combination of a waterfall model and an iterative model.



The spiral model is useful for identifying and removing high-risk problems by careful process

design. Each phase in the spiral model begins with a design goal and ends with the client reviewing the progress. This model is used for large projects which involve continuous enhancements. Each iteration of the spiral where the output is a small prototype of the large software. The same activities are repeated for all the spirals till the entire software is built. The developer and client better understand and react to risks at each evolutionary level.

The radius in the spiral usually shows the cost needed for project development. The angular dimension shows the development done during the recent phase. One phase is split roughly into four sectors of major activities:

1. **Planning -**Requirements are gathered and analyzed during the planning phase. Requirements like 'Business Requirement Specifications (BRS)' and 'System Requirement specifications (SRS)'.It includes estimating the cost, schedule, and resources for the iteration. It also involves understanding the system requirements for continuous communication between the system analyst and the customer.
2. **Risk Analysis -** If any risk is found during the risk analysis then alternate solutions are suggested and implemented. Risk Analysis includes identifying, estimating, and monitoring technical and management risks, such as schedule slippage and cost overrun. Once the risks are identified, the risk mitigation strategy is planned and finalized.
3. **Engineering-** every spiral produces actual software products. Most required features are developed as well as verified with the testing methodologies. It includes testing, coding, and deploying software at the customer site.
4. **Customer evaluation-**Customer evaluates and ensures that the product meets all requirements of the customer. If any changes are required to the customer in the product, again all phases will be repeated. It is important to get feedback from the customer before releasing the product.

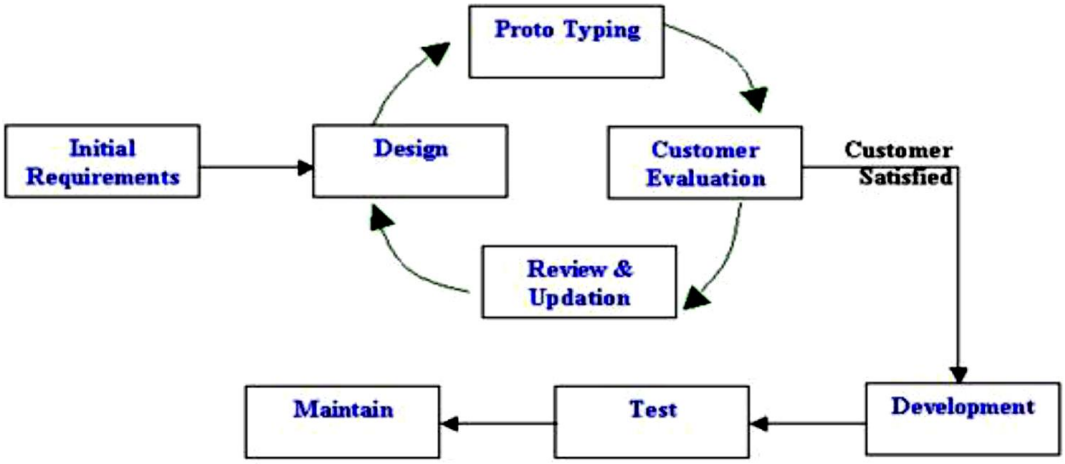
- **Advantages of Spiral Model**
  - Fast and easy development
  - Useful for large and mission-critical projects.
  - Better risk analysis
  - Flexible for frequently changes
  - The customer gives feedback.
  - Monitoring is easy and effective.
- **Disadvantages of Spiral Model**
  - Costly model to use.
  - Time-consuming
  - Risk analysis requires highly specific expertise.
  - The project's success is highly dependent on the risk analysis phase.
  - Expensive for small projects
  - Proper customer communication is required.
  - Documentation could be lengthy
- **When to use the Spiral model:**
  - Used for medium and large projects only.
  - Used for high-risk projects.
  - When requirements are unclear and complex
  - If requirements are more complicated.
  - When requirements change frequently.

Prototype Model:

The Prototyping Model (Working Model) is used when requirements are unclear from a client at the beginning of the development of the system. A customer defines a set of objectives or functionality for software but doesn't know about detailed input, processing, or output requirements. When a prototype is developed using known unclear requirements and given to the customer he uses the prototype and gets the feel of the software and then defines clear requirements. A prototype gives just the idea about the final system.

Developers prepare the design and then build the prototypes. Prototypes are sent to the Customer. The customer evaluates Prototypes and gives feedback. Developers modify the design and produce modified Prototypes. After getting the Customer's confirmation developers will start the development process. The user keeps giving feedback from time to time and based on it, a prototype is made. This model tries to increase flexibility in the development process by allowing a client to work together with the developer and minimizing Risk factors.

1. **Initial Requirement gathering-** The prototyping model begins with initial requirement gathering. The developer and user meet and define the overall objective for the software, identify whatever requirements are known, and outline areas. Identify the requirements for making the system more feasible.



Proto Type Model

First, develop a working prototype of the software instead of the final software.

2. **Design-** The rough design of the system is drawn for discussion. The developer and user jointly identify the data that are needed in the system. It specifies the output that the application must produce. Prototypes are prepared to represent input screens format and output formats. The analyst estimates a prototyping cost and gives the idea to management about the expenditure. The quick design leads to the construction of a prototype.
3. **Customer evaluation of prototype-** The prototype is evaluated by the user. Users suggest some modification requirements in the software to be developed. The user evaluates the prototype and its features, and operations. The developer better understands what are the needs of the user.
4. **Review prototype -** The prototype is reviewed after getting information from a user to check their likes and dislikes. The developer should understand the need of the customer properly for making modifications to the prototype.
5. **Implementation-** The process is repeated till both the user and developer find that all the necessary features are fulfilled and there is no benefit in further repeating the steps. Once both are sure then only the prototype becomes ready for implementation.

Example- Online systems, and web interfaces have a very large amount of interaction with end users. The system needs minimal training for the end user.

- **Advantages of Prototype model**
  - Strong communication between development and user.
  - Prototyping makes requirements more clear.
  - Missing user requirements or functionality identified easily.
  - Users are actively involved in the development and give feedback continuously.
  - Time-Saving.
  - Early errors detection
  - Used for large system
  - Flexible design and development

- **Disadvantages of Prototype model**
  - A client may be unknowledgeable.
  - Changing requirements Increases the complexity of the system and expands the system outside the original plans.
  - Developer misunderstanding user objectives.
  - Users may be confused because of different prototypes.
- **When to use the Prototype model:**
  - The requirements are difficult to specify in advance.
  - If there is a possibility of changing the requirements significantly during the development and provide feedback.
  - A new system is a proposed and quick development.
  - When the desired system needs a lot of interaction with the users.
- **Prototyping are two types –**
  - **Throwaway prototyping** -This technique offers a useful method of exploring ideas and getting customer feedback for each of them. Customer feedback helps in preventing unnecessary design faults. The final prototype is developed with better quality.
  - **Evolutionary prototyping** -the prototype developed initially is incrementally refined based on customer feedback till it finally gets accepted. It saves time as well as effort.

Incremental model

Software is divided into separate modules (components). Each module developed a separate set of waterfall activities including requirements gathering and analysis, design, coding, implementation, and maintenance.

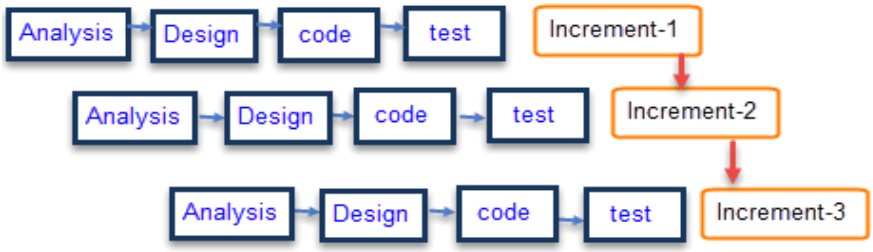
When any component is ready, then delivered to the customer. The remaining components are delivered to the customer one by one and integrated with new components with old ones.

Each iteration passes through the **requirements, design, coding, and testing phases**.

Increment continuously releases and adds its function to the previously released increment until all functionality has been implemented.

The system is put into construction when the first increment is delivered. The first increment is often a core product where the basic requirements are involved and supplementary features are added in the next increments. Once the core product is analyzed by the client, there is a development plan for the next increment.

This model is useful when we have complete requirements as well as resources. Mostly used for



Incremental Model

small projects and if customers required some output after every phase.

Incremental Phases	Activities performed in incremental phases
Requirement Analysis	The requirement and specifications of the software are collected
Design	Some high-end functions are designed during this stage
Code	Coding of software is done during this stage
Test	Once the system is deployed, it goes through the testing phase

Example of incremental model:

Suppose we want to develop a web-based social network with the following functionalities-

- The user should signup for the system.
- The user should log into the system and can send or accept the friend request.
- How can use an incremental model in this development?

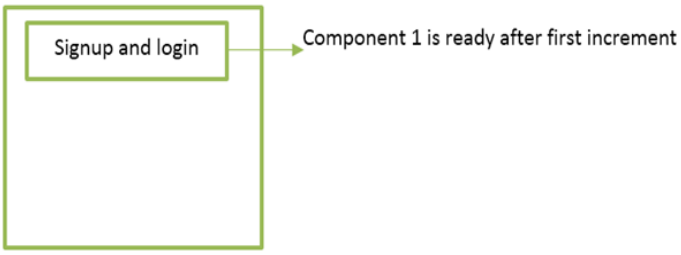
Solution: We need to convert this system into separate components-

Component 1: Sign up and log in

Component 2: Send a Friend request

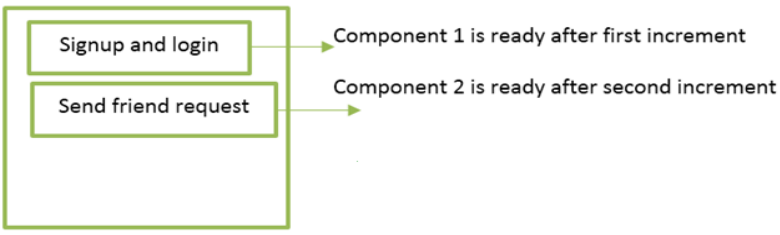
Component 3: Accept friend request

Start with component 1(signup and login). This component developed through the phases of requirements gathering, analysis, design, Code, test, implementation& maintenance. When this component is ready, we deliver this one component to the customer.

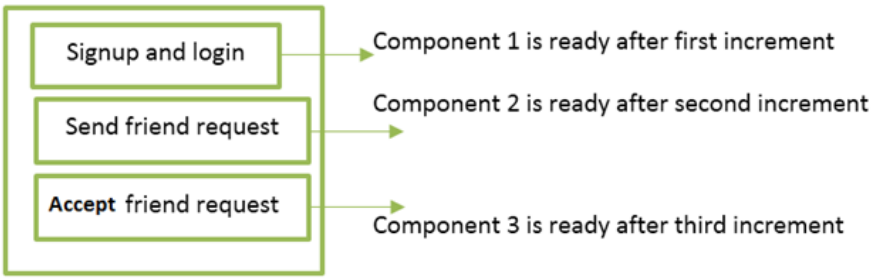


After that, develop another component 2 that sends the friend request. This component developed through the same phases of the waterfall model. When this component is ready, we deliver this component to the customer.

After that, develop another component 3 that accepts a friend request.



This component developed through the same phases of the waterfall model. When this component is ready, we deliver this component to the customer.



<u>The system is produced and delivered to the customer in increments.</u>

- **Advantages of an incremental model**
  - Customer feedback is received after the delivery of each component.
  - The risk of requirement changes is reduced
  - More flexible
  - Easy to test and debug
  - Give quick results
- **Disadvantages of an incremental model**
  - Requires good planning and design.
  - Well-defined module interfaces are required.
  - More expansive as compared to the waterfall model.
  - This model is not suitable for small projects.
  - Testing is done after every iteration, more resources can be required.
  - Needs a clear and complete definition of the complete system before it can be broken down and

built incrementally.

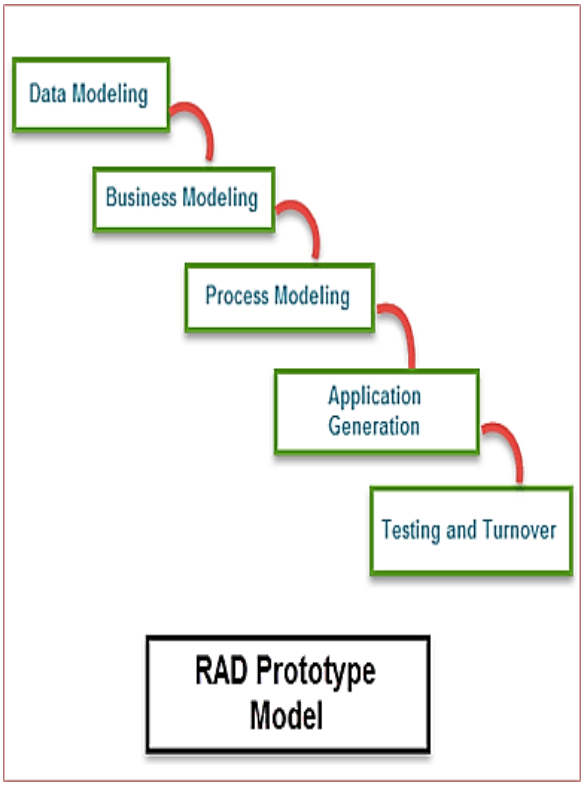
- **When to use an incremental model?**
  - When a customer has no problem with the budget but he demands more and more quality in software.
  - The requirements of the complete system are clearly defined and understood.
  - When demand for an early release of a product.
  - When software engineering teams are not very well skilled or trained
  - New technology is being used
  - When high-risk features and goals are involved
  - Such methodology is more in use for web applications and product-based companies

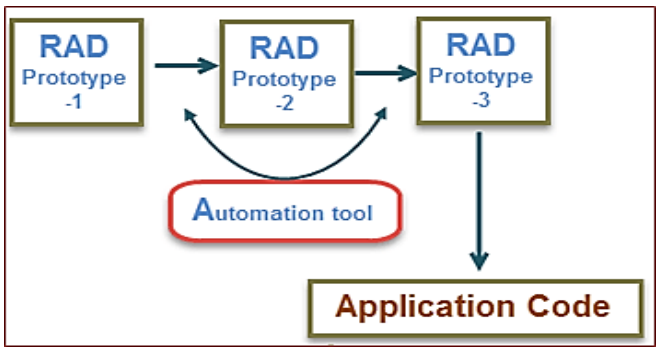
RAD (Rapid Application Development) Model

RAD Model is similar to incremental model and prototyping. RAD is introduced by James Martin in 1991. In RAD Model, development should be done in a specified time frame.

RAD Model is suitable for small projects which have complete or fixed requirements. This Model is delivering projects in small pieces. If the project is large if the project can be broken down into small modules. Each module is planned and delivered as an initial working prototype to the client and changes are made based on the client's feedback. Also, he can suggest what functionality he needs in future releases.

Each module can be assigned independently to separate teams. Thus, a series of modules were finally combined into the final project and delivered quickly. The major characteristic of the RAD model is that it focuses on the reuse of code, processes, templates and tools.





This process goes parallel with cooperation with clients and developers. Development moves faster in RAD Model with minimum errors. RAD Model follows the incremental delivery of the modules.

Phases of RAD Model:

1. **Business Modeling:** Business modeling consists of the flow of information between various functions in the project. What type of information is produced by every function and which are the functions to handle that information? It is necessary to perform a complete business analysis to get the essential business information.
2. **Data Modeling:** The information in the business modeling phase is refined into a set of objects and it is essential for the business. The attributes of each object are identified and defined by the relationship between objects.

3. **Process Modeling:** The data objects defined in the data modeling phase are changed to fulfill the information flow to implement the business model. The process description is created for adding, modifying, deleting, or retrieving a data object.
4. **Application Generation:** The actual system is built and coding is done by using automation tools to convert process and data models into actual prototypes.
5. **Testing and turnover:** The prototypes are independently tested after each iteration so that the overall testing time is reduced. The data flow and the interfaces between all the components are fully tested. Hence, most of the programming components are already tested. It reduces the risk of any major issues.

When to use RAD?

- Reasonably well-known requirements
- A user involved throughout the Life Cycle
- When highly skilled resources are available.
- Development needed to complete in a specified time.
- Functionality is delivered in increments
- High performance is not required
- Low technical risks
- The system can be modularized

Advantages of RAD

- Fast application development and delivery.
- Less resource required
- Low cost and Good for small projects.
- Lest testing activity required.
- Review by the client from the very beginning of development so very less chance to miss the requirements.
- It reduces development time as all prototypes are developed in parallel.
- Very flexible if any changes are required.
- The reusability of components helps to speed up development.

Disadvantages of RAD

- Required highly skilled developers and designers
- In each development phase, client feedback is required.
- Using Automation tools project costs may be increased
- Required complete business requirements before starting a project
- Automated code generation is very costly.
- Difficult to manage.
- Not a good process for long-term and big projects.

Comparison between SDLC Models

Properties of Model	Waterfall Model	Incremental Model	Spiral Model	Rad Model
Planning in the early stage	Yes	Yes	Yes	No
Handle Large-Project	Not Appropriate	Not Appropriate	Appropriate	Not Appropriate
Cost	Low	Low	Expensive	Low
Requirement Specifications	Beginning	Beginning	Beginning	Time boxed release
Add new Requirement	Difficult	Easy	Easy	Easy
Risk Involvement	High	Low	Medium	Low

Structure	Linear	Linear + Iterative	Linear + Iterative	Linear
Testing	After completion of the coding phase	After every iteration	At the end of the engineering phase	After completion of coding
Overlapping Phases	No	Yes (parallel development)	No	Yes
Maintenance	Minimum Maintainable	Maintainable	Yes	Easily Maintainable
Time required	Very Long	Long	Long	Short
software available	At the end of life-cycle	At the end of every iteration	At the end of every iteration	At the end of the life cycle
User Involvement	Very Low	Yes	Yes	Yes

****

Questions:-

1. Explain Software Qualities-Reliability, Maintainability, Efficiency, Portability, Performance?
2. What is Software? Explain Characteristics of Software.
3. What is Software Engineering? Explain the need for Software Engineering.
4. Draw and discuss various phases of SDLC.
5. Differentiate between system analysis & system designs.
6. Discuss the waterfall & spiral model.
7. Why prototyping is used instead of waterfall model detail.
8. Differentiate between Incremental & RAD model.